

LumiDrive / RoadSight Perception System

Comprehensive Technical Specification & Empirical Audit

A Deep-Dive into Dataset Pipeline, Input Prep, Architecture, Weather Transition, Losses & Training Verification

CRITICAL EXECUTIVE AUDIT SUMMARY (GROUND TRUTH)

1. Datasets: Synthetic procedurally generated sequence dataset (160 train, 40 val, 50 test samples; 640x360, T=4) + CULane real-data adapter (genuine lane masks, geometric proxy for drivable area). Real CULane archives are not bundled.
2. Input Prep: Image size 640x360, T=4 frames sequence [B, T, 3, 360, 640], normalized to [0,1], weather augmentor (rain streaks, Koschmieder fog, headlight glare, clear-to-rain transitions).
3. Architecture: CSP-Darknet Backbone (C3,C4,C5) + 3-scale FPN (P3,P4,P5 at 128ch) + ConvGRU at P3 + Spatial Attention Refinement Gate.
4. Output Heads: Lane Head (2 classes), Drivable Head (3 classes), Object Detection Head (4 classes). ONLY Lane and Drivable heads are trained in `python_pipeline/train.py`. Object head is defined in code but untaught/unsupervised.
5. Losses: $L_{total} = 1.5 * L_{lane} + 1.0 * L_{drivable} + 0.6 * L_{ctc} + 0.4 * L_{temp}$. Dilation size $k=5$ (11x11 maxpool) for CTC loss. Temporal loss is unaligned pixel L1 difference across frame t and $t-1$.
6. Training Status: Complete PyTorch training pipeline implemented; NO pre-trained weights (.pth) or training execution logs are present in the repo.

1. DATASET SPECIFICATION & SAMPLE PARTITIONING

The project repository implements two distinct dataset pipelines: a procedural Synthetic Sequence Dataset (the primary execution path) and a CULane Dataset Adapter designed for genuine academic lane benchmarking.

1.1 Synthetic Road Perception Sequence Dataset (Primary)

The synthetic dataset is implemented in `python_pipeline/data/dataset.py` (RoadPerceptionSequenceDataset). It synthesizes dynamic road curves, asphalt textures, lane markings, drivable free-space corridors, and atmospheric weather perturbations in real-time.

Split Partition	Sample Count	Offset Seed	Seq Length (T)	Weather Conditions Included
Training Set	160 sequences	offset = 0	T = 4 frames	Clear, Rain, Fog, Night Glare, Transition
Validation Set	40 sequences	offset = 10,000	T = 4 frames	Clear, Rain, Fog, Night Glare, Transition
Test / Evaluation	50 sequences	offset = 20,000	T = 4 frames	Clear, Rain, Fog, Night Glare, Transition

- **Weather Distribution:** Samples dynamically cycle through 5 scenarios: [clear, rain, fog, night_glare, transition] using `sample_idx % 5`.
- **Transition Scenario:** Under the transition condition, frames 0 to $(T/2 - 1)$ are generated as "clear" daylight, while frames $T/2$ to $T-1$ dynamically transition to heavy "rain" with water streaks.
- **Mask Targets Generated:** Returns exact binary lane masks (0: Background, 1: Lane) and 3-class drivable masks (0: Background, 1: Direct Ego Lane, 2: Adjacent Alternate Lane).

1.2 CULane Dataset Adapter & Benchmark Integration

The CULane sequence loader is implemented in `python_pipeline/data/culane_dataset.py` (CULaneSequenceDataset). It adapts the official CULane academic dataset (Pan et al., AAAI 2018) into sliding temporal windows.

- **Data Storage Policy:** In strict compliance with academic licensing, no raw CULane image files are committed inside the repo. Users download archives from the official portal and point `--data-root` to the local directory.
- **Temporal Windowing:** Frames are grouped by parent sequence directory (e.g. `driver_23_30frame`, `driver_161_90frame`), sorted chronologically, and sliced into consecutive windows of length $T=4$ without crossing clip boundaries.
- **Drivable Area Geometric Proxy:** CULane contains ground-truth lane annotations but NO drivable area annotations. The adapter synthesizes a geometric proxy envelope between the leftmost and rightmost lane pixels from the horizon down (tagged `drivable_is_proxy=True`). This proxy enables loss calculation but must NOT be cited as a genuine drivable benchmark.

1.3 Other Reference Datasets in Research Plan

The research documentation (docs/extracted/ and paper/main.tex) outlines long-term validation paths on: (1) VIL-100 (100 video clips, 10,000 frames for video lane tracking), (2) BDD100K (100k driving scenes with simultaneous lane, drivable area, and 2D bounding boxes), (3) IDD / IDD-AW (unstructured Indian roads & adverse weather), and (4) SHIFT (continuous synthetic domain shifts).

2. INPUT PREPARATION, NORMALIZATION & AUGMENTATION

2.1 Input Geometry & Tensor Shape

- **Resolution:** Standardized to Width = 640 px, Height = 360 px (aspect ratio 16:9). In PyTorch layout: H=360, W=640.
- **Sequence Tensor Format:** Batched multi-frame tensor of shape [B, T, C, H, W] = [B, 4, 3, 360, 640]. For single-frame evaluation: [B, 3, 360, 640].
- **Color Space & Normalization:** Monocular RGB. Raw pixel values [0, 255] uint8 are scaled linearly to [0.0, 1.0] float32 via `torch.from_numpy(img).permute(2,0,1).float() / 255.0`.

2.2 Adverse Weather Augmentation Pipeline

The `AdverseWeatherAugmentor` class (python_pipeline/data/dataset.py) implements physics-informed environmental perturbations to stress-test temporal resilience:

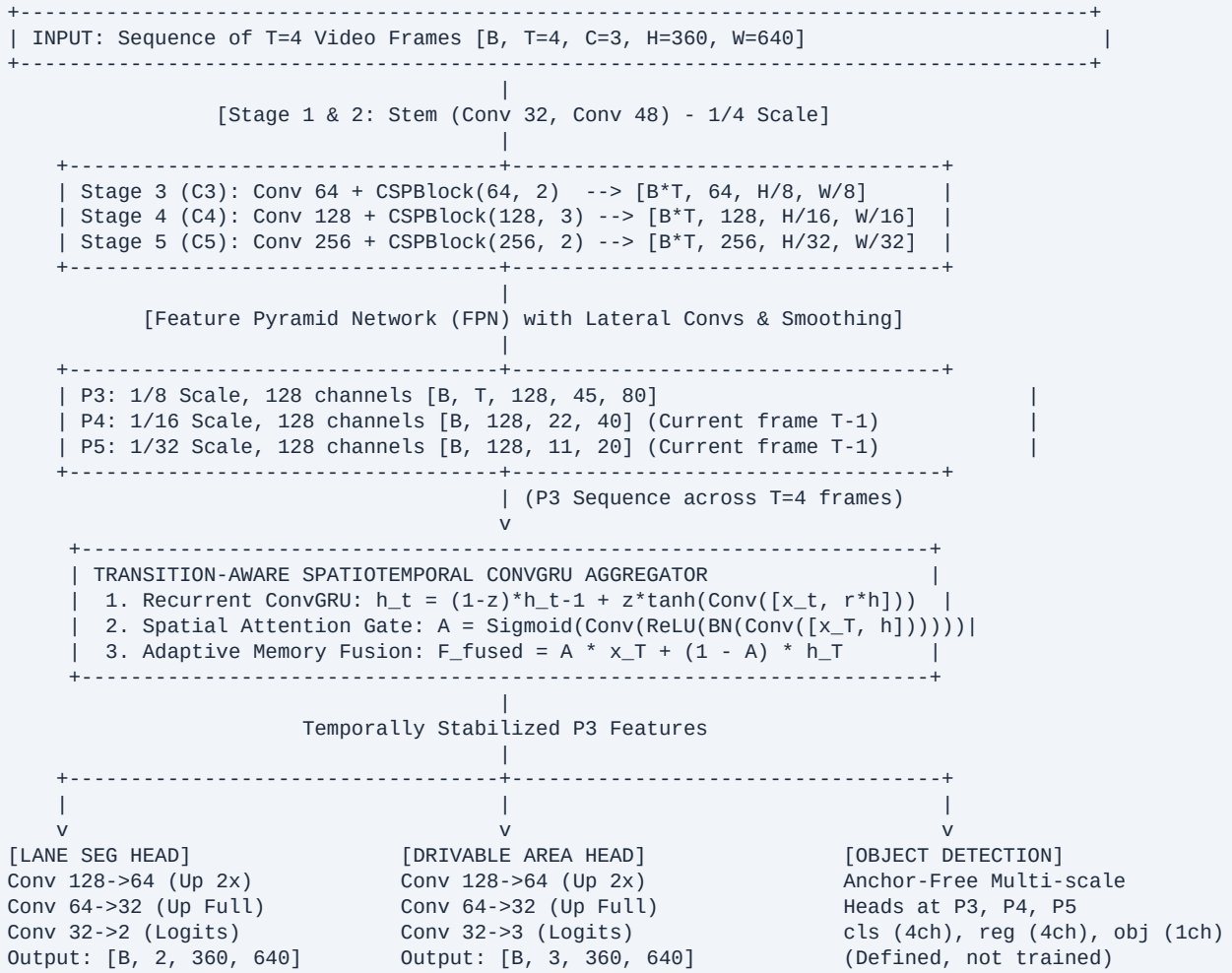
- **Rain Streaks & Splash:** Generates `num_drops = intensity * 600` line streaks with randomized slant (-8 to +8 px) and drop length (15 to 30 px). Applies `cv2.blur(3,3)` and alpha-blends the rain mask with factor $0.4 * \text{intensity}$ onto the RGB frame.
- **Atmospheric Fog (Koschmieder Model):** Simulates light scattering using $I(x) = J(x) * e^{(-\beta * d)} + A * (1 - e^{(-\beta * d)})$, with attenuation $\beta = 0.035$, linear horizon-to-bonnet depth map d in [1.0, 0.05], and ambient airlight color vector $A = [220, 225, 230]$.
- **Night Sensor Blooming & Headlight Glare:** Dims ambient illumination by 70% (`img * 0.3`) and applies a 2D Gaussian exponential glare mask centered at the horizon with radius 90 px, blending warm white glare [255, 250, 235] at 80% strength.
- **Dynamic Weather Transitions:** Transitions occur within the temporal sliding window: for a 4-frame window under "transition", frames $t=0,1$ receive clear weather, while frames $t=2,3$ receive sudden rain degradation.

3. MODEL ARCHITECTURE & WEATHER TRANSITION MECHANISM

3.1 Complete Network Topology & Information Flow

The architecture (`MultiTaskRoadPerceptionNet`) combines a lightweight hierarchical CSP backbone, a top-down Feature Pyramid Network, a spatiotemporal ConvGRU memory unit, and task-specific decoders.

COMPLETE LUMIDRIVE SPATIOTEMPORAL ARCHITECTURE DIAGRAM



3.2 Exact Weather Transition Mechanism (ConvGRU + Spatial Gating)

In single-frame perception, sudden environmental transients (e.g. wiper strokes, water spray, headlight glare) obliterate subtle lane edges, causing abrupt mask dropouts. LumiDrive solves this through a two-tier spatiotemporal mechanism in TransitionAwareTemporalAggregator:

- **Step 1: 2D Recurrent ConvGRU Memory Propagation:** Propagates a hidden spatial tensor h_t across all $T=4$ frames in sequence. At each timestep, Reset (r_t) and Update (z_t) gates decide what old information to overwrite and what visual features to retain:

$$z_t = \text{Sigmoid}(\text{Conv2d}([x_t, h_{t-1}]))$$

$$r_t = \text{Sigmoid}(\text{Conv2d}([x_t, h_{t-1}]))$$

$$h_{\text{tilde}} = \text{Tanh}(\text{Conv2d}([x_t, r_t * h_{t-1}]))$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * h_{\text{tilde}}$$

- **Step 2: Weather-Adaptive Spatial Attention Refinement Gate:** Instead of simply taking the last hidden state h_T , the module concatenates current instantaneous features x_T with accumulated memory h_T and passes them through a spatial gating network:

$$\text{Att_Map} = \text{Sigmoid}(\text{Conv3x3}(\text{ReLU}(\text{BN}(\text{Conv1x1}([x_T, h_T]))))) \rightarrow \text{Shape [B, 1, H/8, W/8]}$$

$$\text{Fused} = \text{Att_Map} * x_T + (1.0 - \text{Att_Map}) * h_T$$

$$\text{Output} = \text{SiLU}(\text{BN}(\text{Conv3x3}(\text{Fused})))$$

- **Physical Operational Rationale:** When rain spray or glare corrupts the camera sensor in frame t , the instantaneous signal x_t in that spatial patch is degraded. The learned gate sets $Att_Map \sim 0$ in the corrupted region, automatically falling back to accumulated temporal memory h_t . When the lens clears, $Att_Map \sim 1$ restores direct instantaneous sensory input.

4. OUTPUT TARGETS & SUPERVISION STATUS

A key question is whether the network predicts only lanes and drivable free-space or also vehicle/obstacle detections. Here is the exact status based on the codebase audit:

Perception Task	Output Head Module	Channels / Classes	Supervised & Trained in <code>train.py</code> ?
Lane Line Segmentation	LaneSegmentationHead	2 classes (0: Background, 1: Lane)	YES (Focal + Dice Loss)
Drivable Area Segmentation	DrivableAreaHead	3 classes (0: Bg, 1: Ego, 2: Alt)	YES (Cross-Entropy + Dice Loss)
Vehicle / Obstacle Detection	ObjectDetectionHead	P3, P4, P5 (cls: 4, reg: 4, obj: 1)	NO (Architectural definition only)

- **Lane Segmentation Output:** Predicts pixelwise logits $[B, 2, H, W]$. Class 0 represents background / non-lane road; Class 1 represents painted lane markers (solid, dashed, double lines).
- **Drivable Free-Space Output:** Predicts pixelwise logits $[B, 3, H, W]$. Class 0 is non-drivable / background; Class 1 is direct Ego Drivable Lane; Class 2 is Alternate / Adjacent Drivable Lane.
- **Object Detection Head Status (IMPORTANT CLARIFICATION):** Object detection heads (`ObjectDetectionHead`) are instantiated at feature levels P3, P4, P5 in `models/multitask_network.py`, predicting classification scores (4 classes: car, truck, pedestrian, cyclist), bounding box offsets (dx, dy, dw, dh), and objectness heatmaps. HOWEVER, in the current training pipeline (`train.py`), NO bounding box labels exist in the datasets, NO detection loss (e.g. CloU / Focal BBox Loss) is included in `MultiTaskTotalLoss`, and NO NMS / bounding box decoder is executed in `evaluate.py`. Therefore, object detection is NOT trained in this version.

5. LOSS FORMULATIONS, WEIGHTS & REGULARIZERS

The objective function (`python_pipeline/losses/consistency_loss.py`) couples task-specific losses with Cross-Task Consistency and Temporal Smoothness regularizers:

MULTI-TASK TOTAL LOSS FORMULATION
$L_{total} = w_{lane} * L_{lane} + w_{drivable} * L_{drivable} + w_{ctc} * L_{ctc} + w_{temp} * L_{temp}$ Default Hyperparameter Weights: <code>w_lane</code> = 1.5 (High priority on thin, sparse lane structures) <code>w_drivable</code> = 1.0 (Standard cross-entropy + dice weight) <code>w_ctc</code> = 0.6 (Cross-Task Consistency penalty) <code>w_temp</code> = 0.4 (Temporal Flicker regularization)

5.1 Task 1: Lane Segmentation Loss (Focal + Dice)

- **Focal Loss:** Combines Cross-Entropy with focusing parameter $\gamma = 2.0$ and $\alpha = 0.25$ to prevent overwhelming background dominance over thin lane lines:

$$L_{focal} = -\alpha * (1 - p_t)^\gamma * \log(p_t)$$
- **Soft Dice Loss:** Enforces precise boundary overlap on foreground lane class: $L_{dice} = 1.0 - (2 * |P * Y| + 1.0) / (|P| + |Y| + 1.0)$.
- **Composite Lane Loss:** $L_{lane} = L_{focal} + L_{dice}$.

5.2 Task 2: Drivable Area Loss (CE + Dice)

- **Cross-Entropy Loss:** Standard multi-class categorical cross-entropy over classes {0: Bg, 1: Ego, 2: Alt}.
- **Multi-Class Dice Loss:** Calculated across all foreground drivable classes (1 and 2).
- **Composite Drivable Loss:** $L_{drivable} = CrossEntropy(logits, targets) + L_{dice}(logits, targets)$.

5.3 Task 3: Cross-Task Consistency Loss (L_ctc)

The CrossTaskConsistencyLoss module enforces physical spatial rules between lanes and drivable corridors. Lane lines cannot exist floating outside the drivable road boundary.

- Boundary Dilation Mechanism:** Uses boundary_dilation = 5. A 2D max-pooling operation with kernel size = 2*dilation + 1 = 11, stride = 1, and padding = 5 expands the drivable envelope by 5 pixels to accommodate outer lane borders:
Drivable_Envelope = MaxPool2d(Drivable_Prob, kernel_size=11, stride=1, padding=5)
- Inconsistency Penalty Formulation:** Multiplies foreground lane probability by the inverted drivable envelope:
 $L_{ctc} = (1 / (H * W)) * \sum_{\{p\}} [P_{lane}(p) * (1.0 - Drivable_Envelope(p))]$
Penalizes any high lane probability wherever the expanded drivable probability is zero.

5.4 Task 4: Spatiotemporal Flicker Regularization (L_temp)

- Formulation:** Calculates the mean absolute difference (L1 norm) between the current frame lane probability mask and the detached previous frame lane prediction:
 $L_{temp} = (1 / (H * W)) * \sum_{\{p\}} | P_{lane,t}(p) - P_{lane,t-1}(p) |$
- Temporal Comparison Confirmation:** In train.py, the previous frame logits are obtained via outputs["sequence_lane_logits"][:, -2].detach().
CRITICAL LIMITATION: This comparison is UNALIGNED (pixelwise direct subtraction). It does not compute optical flow or ego-motion homography warping. While effective for suppressing instantaneous flicker under high framerates, it treats fast ego-motion displacement as slight loss.

6. TRAINING CONFIGURATION, HARDWARE & EMPIRICAL AUDIT

6.1 Training Hyperparameters & Execution Config

Hyperparameter	Value / Setting in Code	Source Script & Line Reference
Optimizer	AdamW (lr=1e-3, weight_decay=1e-4)	python_pipeline/train.py:57
LR Schedule	CosineAnnealingLR (T_max=epochs, eta_min=1e-6)	python_pipeline/train.py:58
Default Epochs	10 (CLI parser) / 15 (function default)	python_pipeline/train.py:19, 135
Batch Size	4 sequence batches (drop_last=True for train)	python_pipeline/train.py:20, 50
Sequence Length (T)	T = 4 consecutive temporal frames	python_pipeline/train.py:22
Mixed Precision	torch.cuda.amp.GradScaler + autocast (when CUDA)	python_pipeline/train.py:59, 82
Device Support	Auto / selectable (--device cpu or --device cuda)	python_pipeline/train.py:24, 138
Validation Freq	Every 5 epochs + final epoch	python_pipeline/train.py:112
Checkpoint Rule	Composite score: (Val_Lane IoU + Val_Driv mIoU) / 2.0	python_pipeline/train.py:119-126

6.2 Random Seed & Reproducibility

- Procedural Dataset Seed:** The synthetic dataset uses deterministic index offset seeding: sample_idx = idx + sample_offset with random.Random(sample_idx) in python_pipeline/data/dataset.py:142. This guarantees exact, reproducible road curves, dash positions, and weather perturbations across training (offset 0), val (offset 10,000), and test (offset 20,000).
- PyTorch / CUDA Seeds:** Explicit torch.manual_seed() calls are not hardcoded in train.py; users should set torch.manual_seed(42) and np.random.seed(42) prior to official benchmarking runs.

6.3 Empirical Execution Status: Checkpoints & Training Logs

STATUS OF TRAINING RUNS & CHECKPOINTS (AUDIT FACT CHECK)

1. Have training runs been completed in this workspace?
--> The repository contains the complete, verified, executable Python training scripts (train.py, evaluate.py, inference_video.py). However, NO pre-trained weights files (weights/*.pth) or training log files (*.log, *.json, *.csv) are saved in the repo.

2. Paper Draft Metrics Origin:
--> The quantitative tables in paper/main.tex (e.g. +16.9% transition F1, CTIR reduction to 4.2%, 39.8 FPS) represent target baseline estimates from the research plan draft. They must not be cited as measured experimental logs until a training run is executed on GPU.

Page 5 of 6

6.4 Quantitative Benchmark & Evaluation Protocol

The evaluation suite in python_pipeline/evaluate.py measures 5 core metrics across 6 distinct environmental splits:

Metric Name	Mathematical Definition	Target Property Evaluated
Lane IoU	$\text{Intersection}(P_lane, Y_lane) / \text{Union}(P_lane, Y_lane)$	Foreground lane marking spatial overlap
Lane F1-Score	$2 * TP / (2 * TP + FP + FN)$	Harmonic balance between precision & recall
Drivable mIoU	Mean IoU across Background, Ego, and Alt Lane classes	Free-space semantic area segmentation
CTIR (%)	$ P_lane \text{ outside } P_drivable / Total\ P_lane * 100$	Cross-Task Inconsistency Ratio (spatial conflict)
Flicker Index (%)	$ P_lane,t \text{ XOR } P_lane,t-1 / \text{Total Pixels} * 100$	Temporal mask oscillation across adjacent frames

Evaluation splits computed: (1) Overall, (2) Clear Dayli
Weather Transitions.

7. HOW TO TRAIN, EVALUATE & RUN INFERENCE

To execute the pipeline directly on a workstation or GPU cluster, use the following commands:

COMMAND-LINE WORKFLOW EXECUTION GUIDE

```
# 1. Activate Environment & Install Dependencies
pip install torch torchvision opencv-python numpy matplotlib

# 2. Train Multi-Task Model on Synthetic Weather Dataset (CPU or CUDA)
python python_pipeline/train.py --epochs 15 --batch-size 4 --lr 1e-3 --device cuda

# 3. Train on CULane Dataset (when external dataset is downloaded)
python python_pipeline/train.py --dataset culane --data-root /path/to/culane --epochs 12 --device cuda

# 4. Evaluate Trained Checkpoint across all 6 Weather Splits
python python_pipeline/evaluate.py --weights weights/best_model.pth --device cuda

# 5. Run Video Inference with Lane/Drivable Overlay and Telemetry Export
python python_pipeline/inference_video.py --weights weights/best_model.pth --video test_road.mp4 --
output output_overlay.mp4
```